

JetBot을 활용한 지능형 RC카 제어



정보컴퓨터공학부

지도교수: 백윤주

팀명: 선넘지마

팀원: 202155520 김도환 dohwan1225@pusan.ac.kr

202255540 김혜은 ww3700@pusan.ac.kr

202155616 천성민 russoll@pusan.ac.kr

1. 서론	3
1.1. 연구 배경 및 필요성	3
1.2. 국내외 관련 기술 동향	3
1.3. 과제의 최종 목표	4
2. 문제 정의 및 요구사항 분석	5
2.1. Push & Clean 과제 시나리오 정의	5
2.2. 시스템 기능적 요구사항	5
2.3. 시스템 비기능적 요구사항	5
3. 시스템 아키텍처 및 세부 구현 알고리즘 설계	5
3.1. 통합 시스템 아키텍처	6
3.2. Perception (인지 모듈) 설계	6
3.3. Decision (판단 및 경로 계획 모듈) 설계	7
3.3.1. 타겟 선정 비용 함수 알고리즘	7
3.3.2 FSM 기반 주행 상태 전환 알고리즘	7
3.4. Control (제어 및 구동 모듈) 설계	8
3.4.1. 양륜 구동(Differential Drive)의 물리적 한계와 제어 목표	8
3.4.2. 오차(Error) 정의 및 모터 속도 맵핑	8
3.4.3. PID 제어기 세부 모델링 및 최적화	8
3.4.4. 실환경 예외 처리 (Anti-Windup 및 LPF)	9
3.5. 실시간 처리 및 지연 시간 최적화	9
4. 개발 환경 및 실험 장비	9
4.1. 하드웨어 스펙 및 구성	9
4.2. 소프트웨어 스택	10
5. 프로젝트 추진 계획 및 역할 분담	10
5.1. 개발 방법론 및 WBS	10
5.2. 월별 세부 추진 일정	10
5.3. 팀원별 상세 역할 분담	11
6. 기대 효과 및 향후 발전 방향	12

1. 서론

1.1. 연구 배경 및 필요성

최근 인공지능 기술의 발전과 함께, 사이버 공간의 지능을 물리적 세계로 확장하는 '피지컬 AI(Physical AI)'가 핵심 연구 분야로 부상하고 있다. 특히 스마트팩토리, 자율주행 물류 시스템, 무인 운반 로봇(AMR) 등 산업 현장에서는 주변 환경을 실시간으로 인식하고 스스로 판단하여 움직일 수 있는 자율 이동 로봇 기술의 중요성이 빠르게 증가하고 있다. 기존 산업용 로봇이 고정된 작업 환경에서 반복 작업을 수행하는 데 초점이 맞춰져 있었다면, 최근에는 변화하는 환경 속에서도 안정적으로 동작할 수 있는 지능형 로봇 시스템에 대한 수요가 확대되고 있다.

본 과제는 이러한 산업적 흐름에 맞추어, 소형 모바일 로봇 플랫폼인 **NVIDIA JetBot**을 활용해 실내 환경에서 무작위로 배치된 목표 물체를 스스로 탐색하고 지정된 구역으로 수집하는 '**Push & Clean**' 시스템을 구현하고자 한다. 이는 실제 산업 현장에서 활용되는 자율 물류 로봇 및 비전 기반 이동 로봇 시스템의 축소형 모델로 볼 수 있으며, 카메라 기반 객체 인식과 실시간 주행 제어 기술을 통합적으로 구현하는 데 목적이 있다.

또한 본 시스템은 단순한 이미지 분류 수준을 넘어, 실시간 영상 처리, 객체 위치 추정, 경로 판단, 모터 제어를 하나의 임베디드 시스템 내에서 동시에 수행해야 한다는 특징을 가진다. 특히 **Jetson Nano**와 같은 제한된 연산 자원 환경에서 AI 추론과 제어 알고리즘을 안정적으로 동작시키기 위해서는 모델 경량화, 지연 시간(Latency) 최소화, 멀티스레딩 기반 병렬 처리 등의 최적화 기술이 요구된다. 이는 실제 산업용 엣지 AI 및 자율주행 시스템 개발 과정에서 중요하게 다루어지는 핵심 요소와 밀접하게 연관된다.

따라서 본 과제는 피지컬 AI 기반 자율주행 로봇 시스템을 직접 설계·구현함으로써, 컴퓨터 비전, 임베디드 AI, 로봇 제어 기술을 통합적으로 학습하고, 실제 산업 현장에서 요구되는 실무형 문제 해결 능력을 향상시키는 데 의의가 있다.

1.2. 국내외 관련 기술 동향

현재 모바일 로봇의 자율 주행 기술은 LiDAR 센서와 SLAM(Simultaneous Localization and Mapping) 알고리즘을 결합하여 주변 환경의 지도를 생성하고 위치를 추정하는 방식이 주류를 이루고 있다. 해당 방식은 높은 정확도를 제공하지만, 센서 가격이 높고 시스템 구조가 복잡하여 소형 로봇이나 저전력 임베디드 플랫폼에 적용하기에는 비용 및 공간 측면의 한계가 존재한다.

이러한 문제를 해결하기 위해 최근에는 단일 RGB 카메라만을 활용하는 비전 기반 자율주행 기술 연구가 활발히 진행되고 있다. 비전 기반 시스템은 하드웨어 구성이 단순하고 소형화에 유리하며, 자율주행 로봇, 스마트 물류 로봇, 무인 운반 차량(AMR) 등 다양한 산업 분야에서 활용 범위가 확대되고 있다. 특히 GPU 기반 엣지 디바이스의 성능 향상과 딥러닝 기술의 발전으로 인해, 카메라 영상만으로도 객체 탐지, 경로 판단, 장애물 회피 등을 수행할 수 있는 수준까지 기술이 발전하고 있다.

객체 인식을 위해서는 YOLO, MobileNet, EfficientDet와 같은 단일 단계 객체 탐지 네트워크가 대표적으로 사용되고 있으며, 최근에는 실시간 처리 성능과 정확도를 동시에

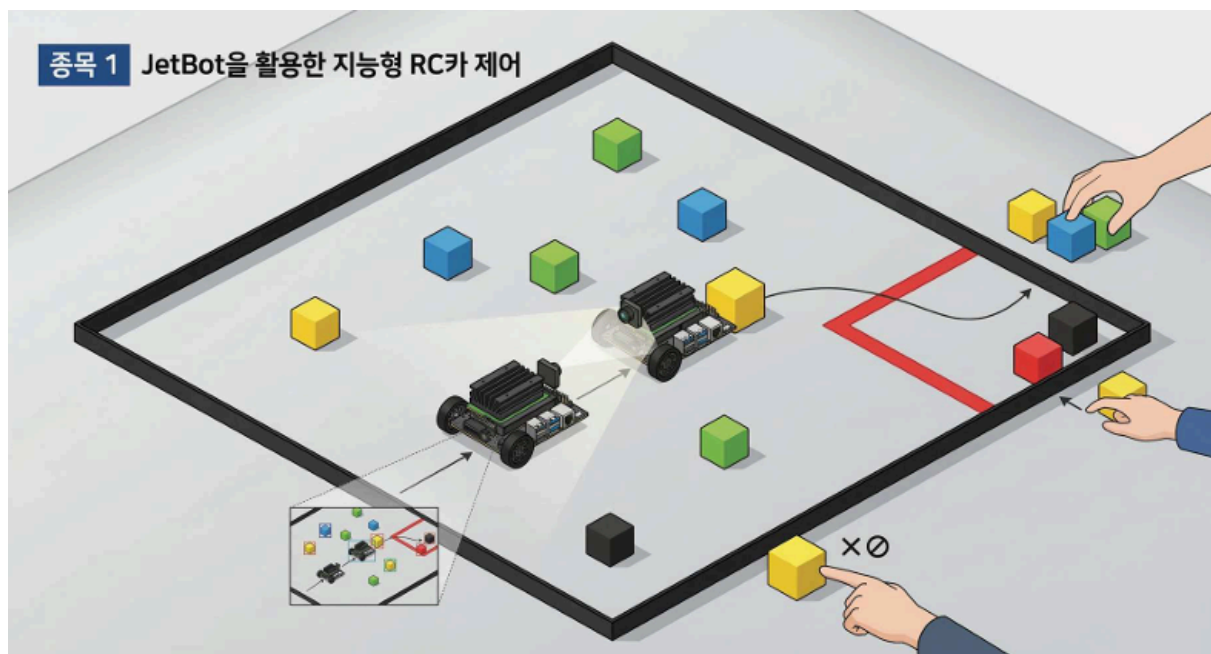
확보하기 위해 경량화된 모델 구조가 주목받고 있다. 또한 제한된 연산 자원을 가지는 임베디드 환경에서는 TensorRT, TensorFlow Lite 등을 활용한 양자화, 프루닝, FP16 최적화 등의 온디바이스 AI(On-device AI) 및 TinyML 기법이 필수적으로 적용되고 있다.

특히 NVIDIA Jetson 계열 플랫폼은 GPU 가속 기반의 실시간 AI 추론이 가능하며, 산업용 자율주행 로봇과 스마트 비전 시스템 분야에서 폭넓게 활용되고 있다. 이에 따라 카메라 기반 객체 인식과 경량 딥러닝 모델을 결합한 임베디드 AI 시스템 설계 능력은 자율주행 및 피지컬 AI 분야의 핵심 기술 역량으로 주목받고 있다.

1.3. 과제의 최종 목표

본 프로젝트의 최종 목표는 NVIDIA JetBot 기반의 지능형 RC카 시스템을 구현하여, 온보드 카메라만을 이용해 경기장 내에 무작위로 배치된 경량 스펀지 블록을 탐지하고, 이를 밀기 방식으로 지정된 목표 구역까지 자율적으로 이동시키는 것이다. 과제는 인지 → 판단 → 제어로 구성된 피지컬 AI 시스템을 실제 하드웨어 환경에서 통합적으로 구현하는 것을 핵심 목표로 한다.

구체적으로는 1.5~2m 크기의 정사각형 경기장 내부에서 검은색 경계선을 인식하며 이동하고, 카메라 영상을 기반으로 목표 블록과 붉은색 목표 구역을 실시간으로 탐지해야 한다. 이후 로봇은 블록의 위치와 이동 방향을 판단하여, 블록을 경기장 밖으로 밀어내지 않으면서 제한 시간 5분 내에 최대한 많은 블록을 목표 구역(30cm × 30cm)으로 이동시키는 자율 주행 기능을 수행해야 한다.



또한 본 프로젝트는 단순 기능 구현에 그치지 않고, 제한된 임베디드 환경에서의 실시간 AI 추론 및 제어 성능 확보를 목표로 한다. 이를 위해 객체 탐지 모델 경량화, 영상 처리 최적화,

PID 기반 주행 제어, 멀티스레드 기반 병렬 처리 구조 등을 적용하여 안정적이고 강건한(**Robust**) 자율주행 시스템을 구현하고자 한다.

최종적으로는 실제 산업 현장에서 활용되는 비전 기반 자율주행 로봇 시스템의 축소형 모델을 구현함으로써, 컴퓨터 비전, 임베디드 AI, 로봇 제어 기술을 통합적으로 설계·구현할 수 있는 피지컬 AI 시스템 개발 역량 확보를 목표로 한다.

2. 문제 정의 및 요구사항 분석

2.1. Push & Clean 과제 시나리오 정의

본 과제는 로봇 팔과 같은 별도의 매니퓰레이터 없이 차체를 이용해 물체를 밀어야(**Push**) 하므로, 최적의 타겟 선정과 정교한 타격 각도 산출이 핵심 문제로 정의된다. 이에 따른 시나리오는 다음과 같다.

- **탐색 및 타겟팅: JetBot이 제자리 회전 주행을**로 주변을 탐색하고, 다수의 스펀지 블록이 식별될 경우 거리, 로봇과의 정렬 상태, 목표 구역과의 직진성을 종합한 비용 함수를 통해 최적의 타겟 1개를 선정한다.
- **사전 정렬(Pre-Align) 및 접근:** 타겟을 향해 단순 직진하는 것이 아니라, 타겟과 목표 구역을 잇는 연장선상으로 이동하여 블록의 후면을 바라보도록 자세를 정렬한 후 접근한다.
- **밀기(Push) 및 사각지대 극복:** 블록이 차체에 닿아 카메라 사각지대로 진입하면, 타겟을 목표 구역(붉은색)으로 전환하고 블록을 밀며 직진한다.
- **복귀:** 블록이 목표 구역에 안착되면 지정된 안전 마진만큼 후진한 후, 다음 타겟 탐색 상태로 복귀한다.

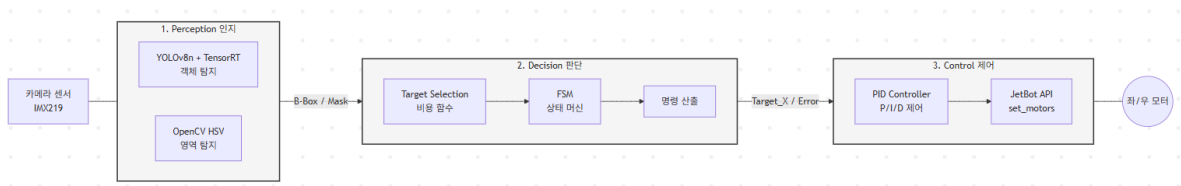
2.2. 시스템 기능적 요구사항

- **실시간 다중 객체 탐지:** 온보드 카메라 프레임 내에서 스펀지 블록, 검은색 테이프, 붉은색 테이프를 **20FPS** 이상의 속도로 동시에 식별 및 좌표를 추출해야 한다.
- **동적 경로 생성 및 PID 제어:** 타겟 블록과 목표 구역의 상대적 픽셀 오차를 기반으로 양륜 모터의 속도를 **PID** 제어하여 조향각을 실시간으로 보정해야 한다
- **안전 및 예외 처리 로직 (Interrupt):** 전면 바닥의 검은색 테이프를 인식할 경우 즉시 구동 모터에 제동을 걸고 후진 및 회전하여 장애 이탈 페널티를 방어해야 하며, 블록을 밀다가 경로를 이탈하면 즉각 밀기를 중단하고 재정렬을 수행해야 한다.

2.3. 시스템 비기능적 요구사항

- **환경 강건성 (Robustness):** 경기장 외부의 바닥 패턴이나 주변 조도 변화에 의해 객체 인식이 교란되지 않도록 HSV 색상 공간 변환 및 적응형 전처리 필터링이 적용되어야 한다
- **자원 효율성 및 추론 가속:** Jetson Orin Nano의 자원 한계와 배터리 전력 소모를 고려하여, AI 비전 모델을 TensorRT 기반의 FP16 정밀도로 가속함으로써 제어 프로세스와의 I/O 병목을 최소화하고 실시간 응답성을 확보해야 한다.

3. 시스템 아키텍처 및 세부 구현 알고리즘 설계



3.1. 통합 시스템 아키텍처

전체 시스템은 Jetson Orin Nano를 코어로 하여 Perception(인지), Decision(판단), Control(제어)의 3계층 비동기 파이프라인으로 구성된다. 카메라 센서로부터 획득한 RGB 이미지는 Perception 모듈로 전달되어 객체의 Bounding Box 좌표와 이진화된 라인 마스크를 출력한다.

이 데이터는 Decision 모듈로 전달되어 타겟을 선정하고 유한 상태 기계(FSM)의 상태를 업데이트하며, 목표 선속도와 각속도를 산출한다.

마지막으로 Control 모듈은 산출된 제어값을 I2C 통신을 통해 모터 드라이버로 전달하고, 양 모터의 RPM을 독립적으로 제어한다. 프레임 지연을 최소화하기 위해 각 모듈은 멀티스레딩 기반으로 병렬 처리된다.

3.2. Perception (인지 모듈) 설계

인지 모듈은 크게 두 가지 파이프라인으로 나뉘며, 연산 병목을 막기 위해 경량화된다.

- **AI 기반 객체 탐지 (스펀지 블록):** 형태와 특징 학습을 위해 YOLOv8n 모델을 사용하며, Jetson Orin Nano의 Tensor Core를 활용하기 위해 FP16 정밀도의 TensorRT 엔진으로 가속한다. 인식된 Bounding Box의 '하단 중심점' 좌표를 추출하여, Y좌표로는 로봇과의 상대적 거리를, X좌표로는 조향 오차를 추정한다.
- **비전 기반 영역 탐지 (경계선 및 목표 구역):** 검은색 테이프와 붉은색 구역은 OpenCV를 활용해 이미지를 HSV 색상 공간으로 변환한 후, Color Thresholding 및 모폴로지(침식/팽창) 연산을 통해 노이즈를 제거한다. 이후 외곽선의 면적과 무게중심을 추출하여 환경 지표로 활용한다.

HSV 색상 공간은 색상(Hue), 채도(Saturation), 밝기(Value)를 분리하여 표현하므로 RGB보다 조명 변화가 있는 환경에서 특정 색상 영역을 안정적으로 분리하는 데

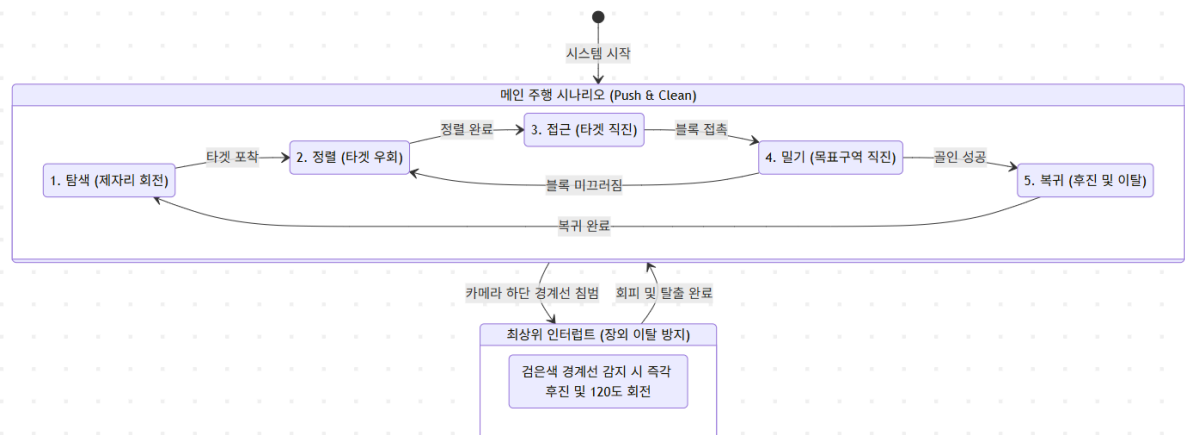
적합하다. 본 시스템에서는 검은색 경계선과 붉은색 목표 구역을 OpenCV 기반 HSV 색상 분할로 탐지한다.

검은색 경계선은 로봇 전방 하단에 주로 나타나므로 영상 하단 영역을 ROI(Region of Interest)로 설정하여 탐지한다. 일정 면적 이상의 검은색 마스크가 검출되면 로봇이 경기장 경계에 근접한 것으로 판단하고, 이 정보는 Decision 모듈의 INTERRUPT_AVOID 상태 전환 조건으로 사용한다.

붉은색 목표 구역은 HSV 공간에서 Hue 값이 0도 부근과 180도 부근에 나뉘어 나타나는 특성이 있으므로, 두 개의 붉은색 범위에 대해 각각 마스크를 생성한 뒤 Bitwise OR 연산으로 결합한다. 이후 모폴로지 연산과 외곽선 검출을 통해 목표 구역의 면적과 무게중심을 계산하고, 검출된 중심 좌표는 STATE_PUSH 단계에서 블록을 밀고 가야 할 기준 방향으로 활용한다.

3.3. Decision (판단 및 경로 계획 모듈) 설계

판단 모듈은 다수의 블록 중 최적의 목표를 고르는 타겟 선정 알고리즘과 로봇의 행동을 결정하는 유한 상태 기계(FSM) 구조를 채택한다.



3.3.1. 타겟 선정 비용 함수 알고리즘

시야에 다수의 블록이 있을 경우, 1) 거리(Y좌표), 2) 정면 정렬 상태, 3) 블록과 목표 구역 간의 일직선 여부, 4) 경계선 근접 위험도를 종합한 비용 함수를 계산하여 가장 점수가 높은 스펀지 1개를 타겟으로 선정한다.

타겟 선정 비용 함수는 여러 스펀지 블록 중 실제로 목표 구역까지 안정적으로 밀기 쉬운 블록을 선택하기 위해 사용된다. 각 블록의 우선순위 점수는 로봇과 블록 사이의 거리, 화면 중심과의 정렬 정도, 목표 구역과의 일직선 여부, 경계선 근접 위험도를 종합하여 계산한다.

$$\text{Score} = w1 \times \text{DistanceScore} + w2 \times \text{CenterScore} + w3 \times \text{GoalAlignScore} - w4 \times \text{BoundaryRisk}$$

여기서 $w1$, $w2$, $w3$, $w4$ 는 각 요소의 중요도를 조절하는 가중치이다. 거리가 가까울수록, 화면 중심에 가까울수록, 목표 구역 방향으로 밀기 쉬울수록 높은 점수를 부여한다. 반대로 블록이 경계선에 가까워 장애 이탈 위험이 큰 경우에는 감점한다. Decision 모듈은 탐지된

모든 블록에 대해 **Score**를 계산하고, 가장 높은 점수를 가진 블록을 현재 타겟으로 선정한다.

3.3.2 FSM 기반 주행 상태 전환 알고리즘

- **STATE_SEARCH**: 카메라 시야에 타겟이 없을 때 제자리에서 회전하며 시야각을 확보한다.
- **STATE_PRE_ALIGN (사전 정렬)**: 타겟을 향해 단순 직진하지 않고, 타겟 블록과 붉은색 목표 구역을 잇는 가상의 연장선을 계산하여 블록의 후면으로 우회 기동한다. 이는 블록을 밀 때 옆으로 미끄러지는 것을 방지하는 핵심 로직이다.
- **STATE_APPROACH**: 정렬이 완료되면 타겟 블록을 화면 중앙에 유지한 상태로 직진하여 접근한다.
- **STATE_PUSH**: 블록과 차체가 접촉하면 타겟을 '붉은색 목표 구역'으로 전환하고 직진한다. 블록이 범퍼에 가려지는 '사각지대' 진입 시, 접촉 직전의 각도와 좌표를 메모리에 저장하여 직진성을 유지하는 추측 방법을 병행한다.
- **INTERRUPT_AVOID (절대 안전 로직)**: 주행 중 검은색 테이프(경기장 경계)가 일정 픽셀 이상 검출되면, 현재의 모든 **State**를 강제 중지하고 즉각 후진 및 **120도** 회전을 수행하여 장애 이탈 페널티를 완벽히 방어한다.

3.4. Control (제어 및 구동 모듈) 설계

3.4.1. 양륜 구동(Differential Drive)의 물리적 한계와 제어 목표

JetBot의 양륜 구동 시스템은 좌우 모터의 회전수 차이를 이용하여 조향한다. 그러나 물리적 직류(DC) 기어 모터의 특성상, 완전히 동일한 PWM 신호를 인가하더라도 1) 모터 내부의 코일 저항 편차, 2) 바닥 면의 비대칭 마찰력, 3) 차체의 무게 중심 편향 등으로 인해 로봇이 직진하지 않고 한쪽으로 휘어지는 정상 상태 오차(Steady-state Error)가 필연적으로 발생한다. 본 과제에서는 이를 보완하고 스펀지 블록을 정확히 타격하기 위해, 카메라 비전 데이터를 폐루프(Closed-loop) 피드백으로 활용하는 이미지 기반 시각 서보잉(**IBVS**) 제어기를 설계한다.

3.4.2. 오차(Error) 정의 및 모터 속도 맵핑

카메라 프레임(해상도 예: 640x480)에서 타겟(스펀지 또는 목표 구역)의 **Bounding Box** 중심 X좌표를 $Target_X$, 화면의 정중앙 좌표를 $Center_X$ (예: 320)로 정의한다. 제어기의 입력이 되는 시간 t 에서의 위치 오차 $e(t)$ 는 다음과 같다.

- $e(t) = Center_X - Target_X$ (오차가 양수이면 타겟이 왼쪽에, 음수이면 오른쪽에 위치함을 의미)

이 오차 $e(t)$ 를 최소화(0으로 수렴)하기 위해 **PID** 제어기 연산을 수행하여 조향 보정값 $U(t)$ 를 산출하며, 이를 로봇의 기본 직진 선속도(**Base Speed**)에 더하거나 빼서 좌우 모터의 최종 출력을 결정한다.

- $MotorLeft = Base\ Speed - U(t)$
- $MotorRight = Base\ Speed + U(t)$

3.4.3. PID 제어기 세부 모델링 및 최적화

산출된 $U(t)$ 는 비례(P), 적분(I), 미분(D) 제어의 합으로 구성되며, 각 게인(Gain)은 주행 테스트를 통해 경험적으로 튜닝된다.

1. 비례 제어 (**P-Control**): $K_p \times e(t)$
 - 오차의 크기에 비례하여 조향력을 강하게 준다. 타겟이 화면 가장자리에 있을수록 빠르게 차체를 회전시켜 타겟을 시야 중앙으로 유도한다.
2. 적분 제어 (**I-Control**): $K_i \times \int e(t) dt$
 - P제어만으로는 해결되지 않는 '모터의 고질적인 편차(정상 상태 오차)'를 누적하여 보상한다. 차가 계속 오른쪽으로 쏠리려는 물리적 특성이 있다면, I제어가 이를 누적하여 왼쪽 모터에 영구적인 추가 가중치를 부여한다.
3. 미분 제어 (**D-Control**): $K_d \times de(t)/dt$
 - 오차의 변화율(기울기)을 계산하여 미래의 오차를 예측한다. 차체가 타겟 중앙을 향해 빠르게 회전하다가 중앙을 지나쳐 반대쪽으로 넘어가는 과조향 및 진동 현상을 억제하여, 마치 브레이크를 잡듯 부드러운 곡선 궤적을 완성한다.

3.4.4. 실환경 예외 처리 (Anti-Windup 및 LPF)

- **Anti-Windup (적분 누적 방지):** 블록을 밀다가 장애물에 걸려 오차가 줄어들지 않는 상황에서, I제어 값이 무한히 누적되어 모터가 폭주하는 현상을 막기 위해 적분값의 최대/최소 한계치를 설정한다.
- **저주파 통과 필터:** 비전 인식 특성상 **Bounding Box** 좌표가 미세하게 떨리는 노이즈가 발생할 수 있다. 이는 미분(D) 제어기에서 순간적인 발산을 유발하므로, 예러 값에 1차 저주파 통과 필터를 적용하여 부드러운 제어값을 모터에 인가한다.

3.5 실시간 처리 및 지연 시간 최적화

본 시스템은 카메라 입력, 객체 탐지, 색상 기반 영역 인식, FSM 판단, PID 제어가 연속적으로 수행되므로 각 단계의 지연 시간이 누적되면 로봇의 반응 속도가 저하될 수 있다. 이를 방지하기 위해 **Perception, Decision, Control** 모듈을 분리하고 멀티스레딩 기반의 비동기 처리 구조를 적용한다.

카메라 입력 스레드는 지속적으로 최신 프레임을 수집하고, **Perception** 모듈은 가장 최근 프레임을 기준으로 YOLOv8n 추론과 HSV 기반 마스크 처리를 수행한다. 오래된 프레임이 큐에 누적될 경우 실제 로봇 위치와 맞지 않는 판단이 발생할 수 있으므로, 처리 지연이 발생하면 최신 프레임을 우선 사용하도록 구성한다.

경계선 감지 정보는 안전과 직접적으로 관련되므로 일반 객체 추적보다 높은 우선순위로 처리한다. 검은색 경계선이 일정 면적 이상 검출되면 현재 **FSM** 상태와 관계없이 **INTERRUPT_AVOID** 로직을 우선 수행하여 장애 이탈을 방지한다.

최적화 결과는 평균 **FPS**, 프레임당 추론 지연 시간, 경계선 및 목표 구역 인식률, **PID** 제어 후 조향 오차를 기준으로 평가한다. 실험 과정에서 실패 사례를 분석하여 **HSV** 임계값, 비용 함수 가중치, **FSM** 상태 전환 조건, **PID** 계수를 조정한다.

4. 개발 환경 및 실험 장비

4.1. 하드웨어 스펙 및 구성

분류	구성 품목	상세 사양 및 용도
Main Board	NVIDIA Jetson Orin Nano	Quad-core ARM A57, 128-core Maxwell GPU, 4GB Memory / 메인 연산
Sensor	Raspberry pi camera module 3	8MP, 160도 FOV 왜곡 방지 렌즈 적용 / 전방 시야 확보
Actuator	DC Gear Motors & Driver	I2C 통신 기반 모터 드라이버 (PCA9685 등) / 양륜 구동 제어
Power	Li-Po Battery	로봇 구동 및 보드 전원 공급용

4.2. 소프트웨어 스택

- **OS:** Ubuntu 18.04 LTS (JetPack 4.6.1 기반)
- **AI Framework:** PyTorch 1.10, Ultralytics YOLOv8, TensorRT 8.2
- **Computer Vision:** OpenCV 4.1.1 (CUDA 가속 빌드)
- **Language:** Python 3.6 (메인 제어 스크립트), C/C++ (일부 임베디드 레벨 최적화 시 활용)

5. 프로젝트 추진 계획 및 역할 분담

5.1. 개발 방법론 및 WBS

프로젝트는 애자일(Agile) 스프린트 방식을 차용하여, 2주 단위로 하드웨어 테스트, 기능 구현,

통합 및 디버깅 사이클을 반복 수행한다. **Git/GitHub**를 활용하여 소스 코드를 형상 관리하고, 이슈 트래킹을 통해 팀원 간의 작업 진행 상황을 투명하게 공유한다.

5.2. 월별 세부 추진 일정

구분	주요 수행 내용	5월	6월	7월	8월	9월
기획/설계	착수보고서 작성, 시스템 아키텍처 설계, 부품 수급	■				
하드웨어	JetBot 조립, OS/패키지 설치, 모터 구동 캘리브레이션		■			
비전/AI	블록 이미지 수집/어노테이션, YOLO 학습 및 TensorRT 변환			■		
판단/제어	State Machine 구현, PID 제어기 설계, 경로 로직 개발			■		
통합/테스트	모듈 통합, 시뮬레이션 및 실 환경 테스트, 엣지 케이스 수정				■	
최종 최적화	랩타임 단축 튜닝, 최종 결과보고서 작성, 실기 대전 준비					■

5.3. 팀원별 상세 역할 분담

- **천성민 (팀장)** - 시스템 통합 및 레벨/AI 최적화: 기존 임베디드 시스템(**TinyML**, 레지스터 레벨 제어) 개발 역량을 바탕으로 **JetBot** 하드웨어의 입출력 제어 병목을 해소하고, **TensorRT**를 활용한 AI 추론 가속 파이프라인 구축 및 최종 모터 **PID** 튜닝을 총괄한다.
- **김도환** - 컴퓨터 비전 및 데이터 엔지니어링 (**Perception**): 다양한 환경 조도와 배경 패턴을 수집하여 객체 인식용 데이터셋을 구축한다. **OpenCV**를 활용한 이미지 색상 공간 변환, 이진화, 노이즈 필터링 모듈을 전담하여 강건한 시야를 확보한다.
- **김혜은** - 의사결정 알고리즘 및 경로 계획 (**Decision & Planning**): 비전 데이터를

입력받아 로봇의 다음 행동을 결정하는 상태 머신(State Machine) 코어 로직을 설계한다.
블록 탐색 동선과 충돌 회피 알고리즘, 효율적인 밀기(Push) 벡터를 계산한다.

6. 기대 효과 및 향후 발전 방향

본 과제의 성공적인 수행을 통해 제한된 리소스를 가진 단일 임베디드 환경에서 영상 처리부터

자율 주행, 정밀 구동까지 통합된 **Physical AI** 시스템의 구축 경험을 확보할 수 있다. 이는 향후

자율주행 자동차, 스마트 팩토리의 무인 운반차(AGV) 및 청소 로봇 산업의 핵심 기술 단위와

일맥상통한다.

향후에는 단일 로봇 제어를 넘어 다수의 **JetBot**을 이용한 군집 제어기술로 발전시키거나, 강화학습 에이전트를 도입하여 로봇이 스스로 최적의블록 수집 경로를 학습하도록 고도화할 수 있을 것으로 기대된다.